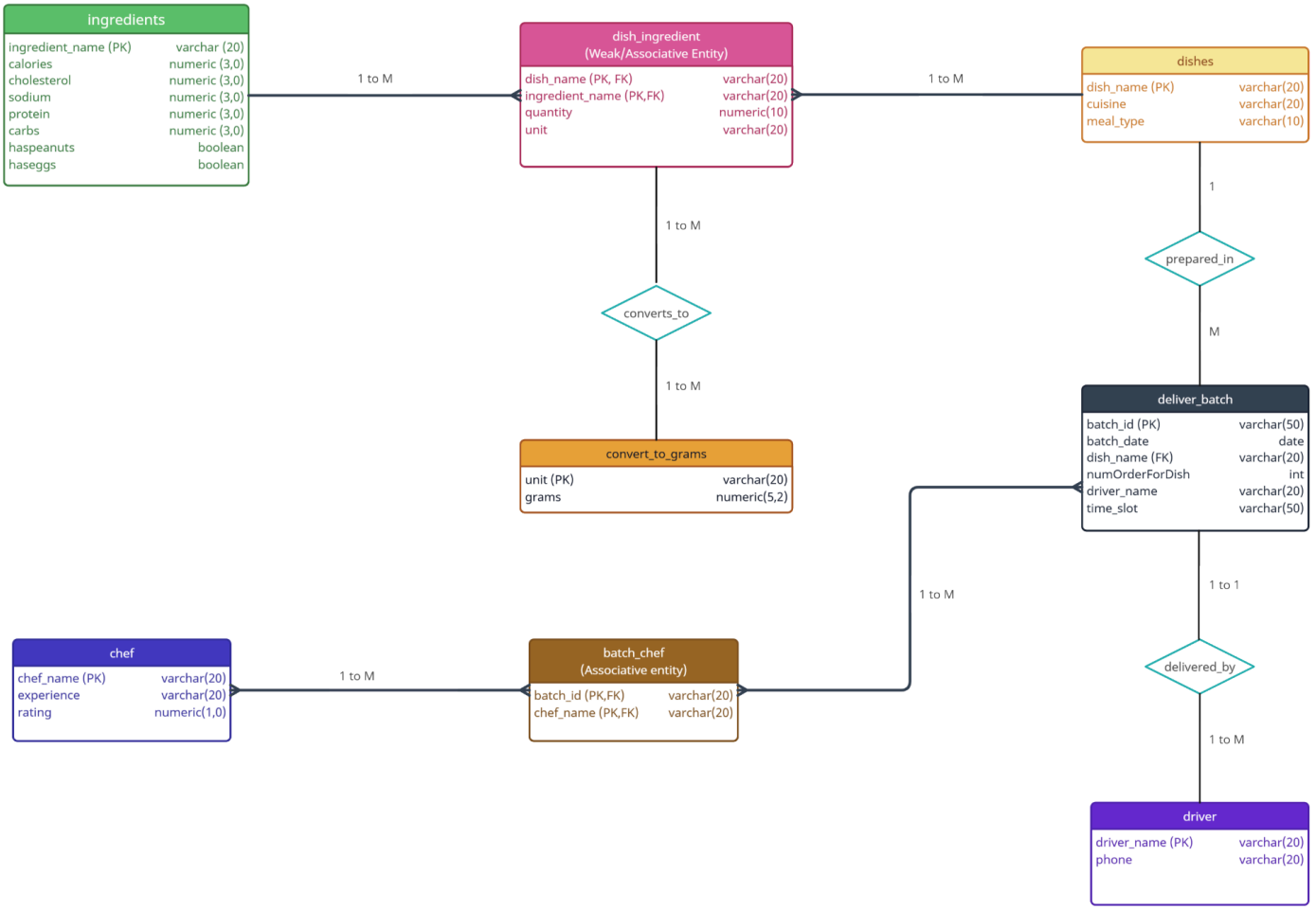


The database represents a catering business of a restaurant. The purpose is to manage the dishes offered by the restaurant, the ingredients used in recipes, nutritional information, delivery batches, drivers, and the chefs who prepared the dishes. While designing the DB, the main focus was to implement proper relationships and, most importantly, reduce redundancy and avoid anomalies during updates.

The main entity in the DB is the dishes table. This entity stores the dishes offered by the restaurant, and it has attributes such as cuisine and meal type (entree, main course, etc.).

Then we have the ingredients table, and since a dish can contain multiple ingredients, and a single ingredient can be used in multiple dishes, their relationship is a many-to-many relationship.

To implement this many-to-many relationship, I decided to create a bridge table called dish_ingredient that serves as an associative entity between the previous tables. The table dish_ingredient is also a weak entity because it is dependent on both the dishes and the ingredients entities. The primary key of dish_ingredient is composed of (dish_name, ingredient_name), which uniquely identifies each ingredient in a specific dish. I have also added attributes like unit and quantity, which serve to describe the relationship itself.

I created a convert_to_grams table to standardize the measurement units, like cups, ounces, and pieces, to grams, since nutritional calculations are based on grams. The relationship between convert_to_grams and dish_ingredient is one-to-many, where one measurement unit can be used in many recipes.

The next table I implemented is deliver_batch, which represents the preparation batches for restaurant orders. Each batch includes an identifier, a preparation date, the dishes being prepared, the number of orders for each dish, the assigned delivery driver, and the delivery time slot. The relationship between dishes and deliver_batch is one-to-many because the same dish may appear in multiple batches.

I created a separate driver entity to store the driver information (name and phone number). This was done to avoid redundancy of driver information in every batch record. The relationship between driver and deliver_batch is one-to-many because one driver can deliver multiple batches, while a batch is assigned to only one driver.

Also, a table named chef was created to represent the chefs who prepare the dishes. Attributes such as years of experience and rating were added to provide more information about the staff. A chef can work many batches, and a batch may involve multiple chefs, so a many-to-many relationship was implemented.

Same as the relationship between dishes and ingredients, I created an associative entity called batch_chef. This table contains foreign keys referencing both chef and deliver_batch, and its composite primary key ensures that each chef-batch assignment is unique.